GPUMODE Lecture Study Notes: Lecture 33
BitBLAS

Core Problem the Lecture Addresses

This lecture studies how to make low precision and mixed precision machine learning computation efficient on real hardware. The central system is **BitBLAS**, a kernel library for low-bit and mixed-precision operators, and **Ladder**, an end-to-end compiler that propagates low-level tensor layouts through machine learning graphs.

The key problem is that modern quantized models increasingly use data formats that hardware does not directly support. A common example is a matrix multiplication where activations remain in FP16 or BF16, while weights are stored as INT4, INT2, or even 1-bit values. The hardware may support tensor cores for FP16 $\times$ FP16, INT8 $\times$ INT8, or newer FP4-style formats, but it usually does not provide a direct tensor core instruction for arbitrary combinations such as

$$\text{FP16} \times \text{INT4} \quad \text{or} \quad \text{FP16} \times \text{INT2}.$$

The lecture therefore asks the following question:

> How can a compiler and kernel library generate high-performance code for data types and precision combinations that are useful for large models, but not directly exposed as native hardware instructions?

The answer has three major parts:

1. Represent arbitrary low-bit data types as packed storage blocks in memory.
2. Convert or reinterpret these custom types into wider hardware-supported types at the correct point in the memory hierarchy.
3. Infer and propagate hardware-aligned layouts so that global memory, shared memory, registers, and tensor core instructions all see the data in the layout they prefer.

Required Background

Quantization and Weight-Only Quantization

Quantization reduces the number of bits used to represent tensor values. In large language models, the most common practical motivation is no longer only faster arithmetic; it is often memory capacity and bandwidth reduction.

For a tensor with N elements stored at b bits per element, the storage cost is

$$\text{StorageBytes} = \frac{Nb}{8}.$$

For example, moving weights from 16-bit storage to 4-bit storage reduces the raw storage by

$$\frac{4}{16} = \frac{1}{4}.$$

This means the model weights use one quarter of the original memory, before accounting for scales, zero points, packing overhead, and metadata.

In **weight-only quantization**, the weights are quantized but activations are often kept in a higher precision such as FP16 or BF16. This is attractive for inference because the weights dominate memory traffic during decode, while activation precision can remain high enough to preserve accuracy.

A typical quantized linear layer may conceptually compute

$$Y = XW^T,$$

where

$$X \in \mathbb{R}^{M \times K}$$

is stored as FP16, while the quantized weight tensor W_q is stored as INT4. The actual value used in computation is recovered by dequantization:

$$W \approx s \cdot (W_q - z),$$

where s is a scale and z is a zero point.

Mixed Precision Computing

Mixed precision means different operands or stages use different numerical formats. The lecture focuses on mixed precision cases where the two matrix multiplication operands have different precisions:

$$A_{\text{FP16}} \times B_{\text{INT4}},$$

or more generally

$$A_\alpha \times B_\beta,$$

where α and β may be different data types.

This is difficult because hardware instructions usually have fixed operand type contracts. A tensor core instruction may require both inputs to be FP16, both to be INT8, or both to follow a specific low-bit floating point format. The compiler must either find a compatible instruction or insert transformations that convert the operands into a supported form.

GPU Memory Hierarchy

A GPU kernel interacts with several memory levels:

- **Global memory**: Large off-chip memory, high bandwidth, high latency.
- **L2 cache**: Shared cache across streaming multiprocessors.
- **Shared memory**: On-chip memory local to an SM, explicitly managed by the program.
- **Registers**: Per-thread storage, fastest but limited.
- **Tensor core operands**: Register fragments arranged in layouts required by matrix multiply instructions.

Efficient kernels must arrange data differently at each level. Global memory wants coalesced access. Shared memory wants bank-conflict-free access. Tensor cores require exact register fragment layouts.

Tensor Cores and Instruction Layouts

A tensor core instruction computes a small matrix multiply-accumulate tile:

$$D = AB + C.$$

A typical instruction shape may be described abstractly as

$$(m_{\text{inst}}, n_{\text{inst}}, k_{\text{inst}}),$$

for example $16 \times 16 \times 16$, although exact shapes depend on architecture and data type.

The hardware instruction expects operands in a specific layout. Even if the mathematical expression is simply

$$C_{ij} = \sum_k A_{ik} B_{kj},$$

the physical register fragments must match the instruction format. This is why a compiler must reason about layout, not only about arithmetic.

Main Concepts

Why Low Precision Is Becoming More Important

The lecture begins from the observation that model precision is trending downward. Modern model repositories contain many 4-bit, 2-bit, and even 1-bit model variants. This is driven by large model deployment constraints:

- Model checkpoints are large.
- GPU memory must also hold activations, key-value cache, temporary buffers, and runtime metadata.
- Decode-time inference is often memory bandwidth bound.
- Reducing weight bandwidth can directly improve throughput.

For a linear layer, the arithmetic work is approximately

$$\text{FLOPs} = 2MNK.$$

The weight memory traffic is approximately

$$\text{Bytes}_W = \frac{NKb_W}{8},$$

where b_W is the number of bits per weight. Reducing b_W lowers memory traffic and can improve performance when the operation is bandwidth-bound.

Three Challenges in Mixed Precision Low-Bit Computing

The speaker identifies three major challenges.

Challenge 1: Unsupported Precision Formats

Many useful formats are not directly represented in hardware or software systems. Examples include custom INT4, INT2, 1-bit, and specialized floating point formats. Standard libraries may not provide a first-class type for these formats.

Challenge 2: Missing Mixed Precision Instructions

Even when a GPU supports low precision tensor cores, it often supports them only for matching operand formats. For example, an instruction may support

$$\text{INT8} \times \text{INT8},$$

but not

$$\text{FP16} \times \text{INT4}.$$

The compiler must therefore map an unsupported mixed precision expression into a sequence of supported transformations and instructions.

Challenge 3: Combinatorial Explosion

The number of possible combinations is large:

$$\{\text{FP32}, \text{FP16}, \text{BF16}, \text{FP8}, \text{INT8}, \text{INT4}, \text{INT2}, \text{INT1}\} \times \{\text{same set}\}.$$

For each combination, a developer may need different packing, decoding, layout, instruction selection, and scheduling strategies. Hand-writing all kernels is not scalable.

Key Insight 1: Memory Can Store Any Data Type

Memory does not inherently understand INT4, INT2, or custom formats. It stores bits. A low-bit tensor can be represented as packed blocks of a wider type such as uint32.

For INT4, one 32-bit word stores eight values:

$$\frac{32}{4} = 8.$$

For INT2, one 32-bit word stores sixteen values:

$$\frac{32}{2} = 16.$$

For 1-bit values, one 32-bit word stores thirty-two values:

$$\frac{32}{1} = 32.$$

The compiler can treat the physical storage type separately from the logical data type.

Key Insight 2: Custom Types Can Be Locally Converted

A custom low-bit value can usually be converted locally into a wider standard type. For example,

$$\text{INT4} \longrightarrow \text{FP16},$$

or

$$\text{INT4} \longrightarrow \text{FP32}.$$

After conversion, the computation can use existing hardware instructions.

The core design choice is **where** to perform this conversion:

- In global memory before the main kernel.
- While loading from global memory to shared memory.
- While loading from shared memory to registers.
- Directly in registers immediately before the tensor core operation.

Each choice changes memory traffic, decoding overhead, shared memory footprint, register pressure, and instruction throughput.

Key Insight 3: Layout Is a First-Class Optimization

Existing compilers such as TVM and Triton can separate computation from scheduling, but the lecture argues that this is not enough for low-bit mixed precision kernels. High-performance tensor core code depends heavily on data layout.

The same mathematical tensor may need different layouts at different memory levels:

$$\text{Layout}_{\text{global}} \neq \text{Layout}_{\text{shared}} \neq \text{Layout}_{\text{register}}.$$

The compiler therefore needs layout-aware tensor abstractions, not just scalar element types.

BitBLAS and Ladder System Overview

BitBLAS

BitBLAS is a kernel library for low-bit and mixed-precision operators. It exposes relatively simple APIs while hiding:

- tensor layout transformations,
- bit packing and unpacking,
- fast dequantization,
- tensor core instruction selection,
- dynamic-shape dispatch,
- tuning database management.

The lecture mentions that BitBLAS has been used in systems such as efficient quantization-aware training, AutoGPTQ-style workflows, and vLLM-related deployment settings.

Ladder

Ladder is an end-to-end compiler. It takes model-level graphs, for example from an ONNX-like input, and applies graph-level layout propagation and fusion. Ladder can reason across operators rather than optimizing each kernel in isolation.

A simplified compiler pipeline is:

$$\text{Model} \to \text{Graph} \to \text{TensorTileGraph} \to \text{LayoutPropagation} \to \text{KernelGeneration} \to \text{OptimizedExecution}.$$

The important idea is that if a layout transformation is needed for one operator, the compiler may propagate that layout backward or forward through adjacent operators to avoid redundant memory transformations.

Tile Language

The lecture also introduces a Triton-like language called **Tile Language**. The motivation is that schedule-template-based BitBLAS code can become difficult to extend. Tile Language allows developers to express kernels more directly using tile-level operations such as copying tiles to shared memory, decoding weights, performing GEMM, and writing back outputs.

Tensor Abstractions

TLType

The first abstraction is a general tensor element type, referred to in the lecture as a type abstraction such as `TLType`. It represents custom data types and their conversion rules.

A custom type can be described by:

- a storage representation,
- a logical interpretation,
- a conversion function to a hardware-supported type.

For example, an INT4 type can be represented as:

$$\text{storage}(x) \in \{0, \ldots, 15\},$$

with conversion

$$\text{decode}(x_q) = s(x_q - z).$$

TTile

A `TTile` represents a tensor tile rather than a scalar. It carries information about:

- shape,
- data type,
- storage type,
- layout,
- memory scope.

A tile is the natural unit of GPU optimization because global memory transactions, shared memory tiles, register fragments, and tensor core instructions all operate on structured groups of elements.

Index Map

An **index map** describes how logical tensor indices map to physical storage locations. For a logical tensor $A[i, j]$, a layout function may map

$$(i, j) \mapsto p,$$

where p is a physical offset.

For a row-major matrix with shape $M \times K$, the index map is:

$$p = iK + j.$$

For a more complex swizzled layout, the physical offset may include tiling, permutation, bit-level packing, and bank-conflict-avoidance transformations.

Tensor Transformation Primitives

The lecture mentions primitives such as:

- `slice`,
- `convert`,
- `pad`,
- `transform_layout`.

These primitives manipulate tiles and layouts. For example:

$$T_{\text{new}} = \text{transform\_layout}(T_{\text{old}}, f),$$

where f is an index map transformation.

Hardware-Level Explanation

Why Memory Layout Matters

A high-performance tensor core GEMM is not merely a loop nest. It is a carefully coordinated data movement pipeline:

GlobalMemory → SharedMemory → Registers → TensorCores → Registers → GlobalMemory.

Each stage has different constraints.

Global Memory

Global memory prefers coalesced access. Threads in a warp should access contiguous or regularly aligned addresses so that the memory system can combine requests into large transactions.

If thread t reads address

$$\text{addr}(t) = \text{base} + t \cdot s,$$

then the best case is often $s = \text{sizeof(element)}$, producing contiguous access.

Shared Memory

Shared memory is divided into banks. If multiple threads in a warp access different addresses mapping to the same bank, bank conflicts occur. A simplified bank index model is:

$$\text{bank}(a) = \left\lfloor \frac{a}{w} \right\rfloor \bmod B,$$

where a is the byte address, w is the bank width, and B is the number of banks.

A layout that is good for global memory may cause bank conflicts in shared memory. Therefore, shared memory often uses a swizzled layout.

Registers and Tensor Core Fragments

Tensor core instructions expect matrix fragments in fixed register arrangements. If the register layout is wrong, extra permutation instructions are needed, or the instruction cannot be used.

Therefore, a compiler must ensure that the path from global memory to registers produces the correct operand fragment layout.

NVIDIA Swizzling

The lecture emphasizes that NVIDIA kernels often use carefully designed swizzling. Swizzling changes the physical placement of tensor elements so that memory access is efficient across levels of the hierarchy.

A conceptual swizzle can be written as:

$$(i, j) \mapsto (i', j'),$$

where i' and j' are transformed indices chosen to improve access patterns.

For low-bit data, swizzling is even harder because layout interacts with packing. Moving one logical value may require moving bits inside a packed word.

Visual Concept

Draw three rows labeled global memory, shared memory, and register fragments. Show the same logical matrix tile being rearranged at each level. Mark global memory as coalesced, shared memory as bank-conflict-free, and registers as tensor-core-compatible.

Mathematical Reasoning

GEMM Computation

The basic GEMM is:

$$C = AB,$$

where

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}, \quad C \in \mathbb{R}^{M \times N}.$$

Each output element is:

$$C_{ij} = \sum_{k=0}^{K-1} A_{ik} B_{kj}.$$

The arithmetic work is:

$$\text{FLOPs} = 2MNK.$$

Arithmetic Intensity

Arithmetic intensity is:

$$\text{AI} = \frac{\text{FLOPs}}{\text{BytesMoved}}.$$

For a matrix multiplication, if we ignore cache reuse and count each matrix once, a rough estimate is:

$$\text{AI} = \frac{2MNK}{\text{bytes}(A) + \text{bytes}(B) + \text{bytes}(C)}.$$

For quantized weights:

$$\text{bytes}(B) = \frac{KNb_B}{8},$$

where b_B is the number of bits per weight. Lowering b_B increases arithmetic intensity if the compute work remains similar.

Roofline Model

The roofline performance bound is:

$$P \leq \min\left(P_{\text{peak}},\ B_{\text{mem}} \cdot \text{AI}\right),$$

where:

- P is achieved performance.
- P_{peak} is peak compute throughput.
- B_{mem} is memory bandwidth.
- AI is arithmetic intensity.

Decode-stage LLM inference is often memory-bound because M is small, commonly $M = 1$, so the same weights are streamed for a small amount of computation. Prefill is more compute-intensive because M or sequence length is larger, enabling more reuse and more arithmetic per byte moved.

Quantized Weight Reconstruction

For affine quantization:

$$w \approx s(q - z),$$

where:

- q is the quantized integer value,
- s is the scale,
- z is the zero point,
- w is the reconstructed floating point value.

For groupwise quantization, the scale depends on the group:

$$w_{g,r} \approx s_g(q_{g,r} - z_g).$$

Here g indexes the quantization group and r indexes the element within the group.

Groupwise scaling complicates layout propagation because scales are shared across groups. Arbitrary permutations of weights may not have a simple matching permutation for scales without additional inverse transformations.

Packed Low-Bit Indexing

For b-bit weights packed into 32-bit words, each word stores

$$P = \frac{32}{b}$$

values.

For logical element index e, the packed word index is:

$$w_{\text{idx}} = \left\lfloor \frac{e}{P} \right\rfloor .$$

The bit offset inside the word is:

$$o = b(e \bmod P).$$

The extracted value is:

$$q_e = (\text{word}_{w_{\text{idx}}} \gg o) \,\&\, (2^b - 1).$$

For INT4, this becomes:

$$P = 8, \quad o = 4(e \bmod 8), \quad q_e = \left(\text{word}_{\lfloor e/8 \rfloor} \gg 4(e \bmod 8)\right) \,\&\, 15.$$

Instruction Selection and Tensorization

The BitNet Instruction Matching Idea

The lecture describes an instruction matching system that decides which hardware instruction should be used for a given tensor expression. The compiler needs to answer:

> Can this computation be mapped to a tensor core instruction, and if yes, how should its loops map to instruction dimensions?

For a matrix multiplication, the mapping is straightforward:

$$(i, j, k) \mapsto (m, n, k).$$

For convolution, the mapping is more complex because convolution contains spatial loops and reduction loops.

A convolution may be written as:

$$Y[n, o, h, w] = \sum_{c,r,s} X[n, c, h + r, w + s] W[o, c, r, s].$$

The compiler must map some loops to the matrix M, N, and K dimensions of a tensor core instruction. This often reduces convolution to an implicit or explicit image-to-column transformation.

Iterator Classification

The lecture describes classifying iterators into categories such as:

- spatial iterators,

- reduction iterators,
- batch-like iterators,
- output-channel iterators.

Then the compiler searches for a valid mapping from computation loops to hardware instruction loops. If successful, the computation can be tensorized.

Example: Convolution to Matrix Multiplication

The convolution expression:

$$Y[n, o, h, w] = \sum_{c,r,s} X[n, c, h + r, w + s]W[o, c, r, s]$$

can be transformed into:

$$Y_{\text{mat}} = X_{\text{im2col}} W_{\text{mat}}^{T}.$$

The flattened dimensions may be:

$$M = N_{\text{batch}} H_{\text{out}} W_{\text{out}},$$

$$K = CRS,$$

$$N = O.$$

This lets tensor cores execute convolution as a GEMM-like operation.

Visual Concept

Draw a convolution loop nest on the left with batch, output channel, height, width, input channel, and kernel loops. Draw an arrow to a GEMM tile on the right with M, N, and K. Label the transformation as iterator classification plus tensorization.

Layout Inference and Propagation

Why Searching Layouts Directly Is Hard

A layout search space can be enormous. Layout choices interact with:

- tile sizes,
- warp-level mappings,
- vectorization width,
- shared memory swizzling,

- tensor core fragment layout,
- packed low-bit storage,
- dequantization location.

A naive search over all possible layouts and schedules is too expensive.

Hardware-Aligned Layout Inference

The lecture describes inferring layouts from the hardware instruction path. Instead of searching arbitrary layouts, the compiler starts from the target instruction requirements and works backward.

Conceptually:

TensorCoreLayout ⇒ RegisterLayout ⇒ SharedMemoryLayout ⇒ GlobalMemoryLayout.

This produces layouts that are naturally aligned with hardware constraints.

Layout Propagation Across Operators

Once an operator prefers a layout, the compiler can propagate that layout through neighboring operators.

For an elementwise operator:

$$Y = f(X),$$

if X is stored in layout L, then Y can often be stored in the same layout:

$$L_Y = L_X.$$

For a chain:

$$X \to \text{cast} \to \text{multiply} \to \text{add} \to Y,$$

the compiler may fuse the operators and keep the same layout in registers.

Three Categories of Layout Transformations

The lecture categorizes layout propagation cases into three broad types.

Linear Transformations

These are layout transformations that preserve information and can be represented as index transformations. A simple permutation is:

$$(i, j) \mapsto (j, i).$$

More complex swizzles may still be invertible.

Compression-Preserving Transformations

Low-bit packing changes the representation but still preserves one-to-one logical information if interpreted correctly. For example, INT4 packing maps eight logical values into one 32-bit word.

Propagation must account for bit packing:

$$\text{logical index} \rightarrow \text{packed word} \rightarrow \text{bit offset}.$$

Groupwise Scaling Transformations

Groupwise scaling can lose layout propagation information because multiple values share a scale. If weights are permuted arbitrarily, the scale tensor may not transform in the same way.

For group size G, the group index of element e is:

$$g = \left\lfloor \frac{e}{G} \right\rfloor.$$

If a layout transformation changes element order across groups, the scale mapping becomes nontrivial.

Inverse Layout Transformations

The lecture mentions using an automatic differentiation-like method to construct inverse layout transformations. Given a layout expression, the compiler builds a syntax tree and propagates inverse relationships backward.

If the forward layout is:

$$y = f(x),$$

the compiler wants an inverse:

$$x = f^{-1}(y).$$

For simple affine layouts:

$$y = ax + b,$$

the inverse is:

$$x = \frac{y - b}{a}.$$

For more complex layout expressions involving floor division, modulo, and bit operations, the compiler uses structured rules over the expression tree.

Where Should Dequantization Happen?

The Three Main Choices

For quantized GEMM, the system must decide where to convert W_q into a wider type.

Option 1: Dequantize in Global Memory

A separate kernel dequantizes weights into global memory:

$$W_{\mathrm{FP16}} = \mathrm{dequantize}(W_q).$$

Then the GEMM kernel reads W_{FP16}. This reduces decode overhead inside the GEMM, but it loses memory savings during the GEMM because the weight tensor is now wider.

Option 2: Dequantize in Shared Memory

The kernel loads packed low-bit weights from global memory, dequantizes them, and stores the expanded values in shared memory.

This saves global memory bandwidth:

$$\mathrm{GlobalBytes} \approx \frac{KNb_W}{8},$$

but shared memory now stores expanded values.

Option 3: Dequantize in Registers

The kernel keeps global memory and shared memory in packed low-bit form, then dequantizes only after loading into registers.

This maximizes memory savings:

$$\mathrm{GlobalBytes} \downarrow, \quad \mathrm{SharedBytes} \downarrow .$$

However, each register load may require decoding work. This can increase instruction overhead and register pressure.

Latency-Oriented Policy

The lecture describes a latency-oriented policy for deciding where dequantization should occur. The trade-off is:

$$\mathrm{TotalTime} \approx \mathrm{MemoryTime} + \mathrm{DecodeTime} + \mathrm{ComputeTime}.$$

If the operation is memory-bound, reducing memory traffic is valuable:

$$\mathrm{MemoryTime} = \frac{\mathrm{BytesMoved}}{B_{\mathrm{mem}}}.$$

If the operation is compute-bound, extra decode instructions may hurt performance more than memory savings help.

Fast Dequantization

Naive Bit Extraction

A naive INT4 decode path extracts one value at a time using shifts and masks.

```
uint32_t packed = packed_weights[word_idx];
uint32_t q = (packed >> bit_offset) & 0xF;
float value = scale * (static_cast<float>(q) - zero);
```

This is simple but inefficient because:

- it decodes one value at a time,
- it uses multiple integer operations per element,
- it may require scalar type conversion instructions,
- it does not fully exploit vectorized hardware paths.

Vectorized Fast Decoding

BitBLAS provides fast decoding paths that convert multiple low-bit elements at once. For INT4, one 32-bit word contains eight values. A good decoder should exploit this structure:

$$\text{uint32} \rightarrow 8 \times \text{INT4} \rightarrow 8 \times \text{FP16}.$$

The lecture mentions using architecture-specific instructions and packed conversion logic. The precise generated code may contain low-level PTX-style operations, but the conceptual goal is to reduce the per-element cost of dequantization.

Why Fast Decoding Matters More at Lower Bit Widths

For INT4, one word contains eight values. For INT2, one word contains sixteen values. For 1-bit, one word contains thirty-two values.

As bit width decreases, the number of logical values per packed word increases:

$$P = \frac{32}{b}.$$

If decoding is scalar, the amount of extraction work per word increases. Therefore, fast vectorized decoding becomes increasingly important for 2-bit and 1-bit kernels.

Decode Cost Versus Memory Savings

A simplified performance model is:

$$T = \max\left(\frac{\text{FLOPs}}{P_{\text{compute}}}, \frac{\text{Bytes}}{B_{\text{mem}}}\right) + T_{\text{decode}}.$$

Fast decoding reduces T_{decode}. If the operation is small and memory-intensive, decoding overhead may dominate unless optimized.

Dynamic Shape Code Generation

Why Dynamic Shapes Are Hard

Large language model inference often has dynamic dimensions. In decode, M may be 1. In prefill, M may be a larger sequence length. A single kernel configuration is rarely optimal for all cases.

For GEMM:

$$C_{M\times N} = A_{M\times K} B_{K\times N}.$$

The dimension M may vary at runtime.

Segmented Dispatch

BitBLAS handles dynamic shapes by generating and tuning kernels for different shape segments. For example, it may use different kernels for:

$$M = 1, \quad 1 < M \le 16, \quad 16 < M \le 32, \quad 32 < M \le 64, \quad \ldots$$

A runtime dispatcher chooses the appropriate kernel:

$$\text{kernel} = D(M).$$

Tuning Database

Because tuning every shape from scratch would be expensive, BitBLAS stores tuning results in a database. The tuning process searches candidate tile configurations and records the best result.

A simplified tuning objective is:

$$\theta^* = \arg\min_{\theta\in\Theta} T(M, N, K, \theta),$$

where θ includes tile sizes, pipeline stages, vectorization choices, and dequantization placement.

Code Walkthrough

Representative Packed INT4 Decode

The following code illustrates the basic logic of extracting INT4 values from 32-bit packed storage.

```
__device__ half decode_int4_value(
    uint32_t packed,
    int element_id,
    half scale,
    half zero
) {
    int shift = 4 * (element_id & 7);
    uint32_t q = (packed >> shift) & 0xF;
    half q_half = __float2half(static_cast<float>(q));
    return scale * (q_half - zero);
}
```

Line-by-line hardware interpretation:

- `packed` is a 32-bit word loaded from global or shared memory.
- `element_id & 7` selects one of eight 4-bit lanes inside the word.
- The shift moves the target 4-bit field to the low bits.
- The mask `0xF` keeps only the 4-bit quantized value.
- The value is converted to FP16.
- Scale and zero point reconstruct the approximate real value.

This code is simple but not necessarily fast. A production BitBLAS kernel tries to decode multiple values together and use hardware-specific packed operations.

Representative Quantized GEMM Structure

```
__global__ void quantized_gemm_kernel(
    const half* A,
    const uint32_t* B_packed,
    const half* scales,
    half* C,
    int M,
    int N,
    int K
) {
    // Each block computes one output tile.
    // Details such as swizzling, tensor core fragments,
    // and vectorized decoding are omitted here.
```

```
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    half acc = __float2half(0.0f);

    for (int k = 0; k < K; ++k) {
        int logical_b_index = k * N + col;
        int word_idx = logical_b_index / 8;
        int lane = logical_b_index % 8;

        uint32_t packed = B_packed[word_idx];
        half b = decode_int4_value(
            packed,
            lane,
            scales[col],
            __float2half(0.0f)
        );

        half a = A[row * K + k];
        acc = __hfma(a, b, acc);
    }

    C[row * N + col] = acc;
}
```

This is not a high-performance kernel. It is a conceptual baseline. It has several inefficiencies:

- It performs scalar decoding inside the inner loop.
- It does not use shared memory tiling.
- It does not use tensor cores.
- Memory accesses to packed B may be irregular.
- It uses one thread per output element, which is inefficient for large GEMM.

A production BitBLAS kernel instead tiles the computation:

$$C_{\text{tile}} = A_{\text{tile}} B_{\text{tile}},$$

loads tiles cooperatively, decodes in vectorized form, and uses tensor core MMA instructions.

Representative Tiled GEMM Pseudocode

```
for each block tile C_tile:
    initialize accumulator fragments in registers
```

```
    for k_tile in K dimension:
        load A_tile from global memory to shared memory
        load packed B_tile from global memory to shared memory

        synchronize threads

        decode B fragments from shared memory to registers
        load A fragments from shared memory to registers

        issue tensor core MMA instructions

        synchronize threads

    write accumulator fragments back to global memory
```

Hardware behavior:

- Global memory loads should be coalesced.
- Shared memory layout should avoid bank conflicts.
- Register fragments must match tensor core instruction layout.
- Synchronization is needed after cooperative shared memory loads.
- Accumulators remain in registers across the K-tile loop.

Representative BitBLAS API Sketch

The lecture shows that BitBLAS aims to expose a simple interface. A simplified Python-like usage pattern is:

```python
import bitblas

M = 1
N = 4096
K = 4096

config = bitblas.MatmulConfig(
    M=M,
    N=N,
    K=K,
    A_dtype="float16",
    W_dtype="int4",
    out_dtype="float16",
    accum_dtype="float32",
    group_size=128,
    with_scaling=True,
    fast_decoding=True
```

```
)

operator = bitblas.Matmul(config)
operator.hardware_aware_finetune()
cuda_source = operator.get_source()
```

Important ideas in this sketch:

- M, N, and K define the GEMM shape.
- The activation type and weight type may differ.
- The accumulator type can be wider than the output type.
- Group size controls scale sharing.
- Fast decoding enables vectorized dequantization paths.
- Hardware-aware tuning chooses an efficient schedule.

Representative Tile Language Kernel Sketch

Tile Language is designed to make these kernels easier to express directly.

```
@tile_language.kernel
def matmul_int4_kernel(A, Bq, Scale, C):
    A_shared = alloc_shared((BM, BK), "float16")
    B_shared = alloc_shared((BK, BN_PACKED), "uint32")
    A_frag = alloc_fragment((WM, WK), "float16")
    B_frag = alloc_fragment((WK, WN), "float16")
    C_frag = alloc_fragment((WM, WN), "float32")

    clear(C_frag)

    for ko in range(0, K, BK):
        copy(A[block_m, ko], A_shared)
        copy(Bq[ko, block_n], B_shared)

        pipeline_barrier()

        load(A_shared, A_frag)
        decode_int4(B_shared, Scale, B_frag)

        mma(A_frag, B_frag, C_frag)

    store(C_frag, C[block_m, block_n])
```

Conceptually:

- `alloc_shared` creates shared memory tiles.

- `alloc_fragment` creates register fragments.
- `copy` describes global-to-shared movement.
- `decode_int4` describes low-bit conversion.
- `mma` maps to tensor core instructions.
- `store` writes accumulated results back to global memory.

Performance Intuition

Decode Stage Versus Prefill Stage

The lecture highlights that LLM decode and prefill behave differently.

Decode

In decode, batch size and sequence position may make M small, often $M = 1$. The operation is close to matrix-vector multiplication:

$$y = Wx.$$

The amount of weight data loaded is large relative to compute. Therefore, decode is often memory-bound.

Reducing weight precision from 16-bit to 4-bit can strongly improve decode throughput because it reduces memory traffic.

Prefill

In prefill, many tokens are processed together, so the operation becomes closer to matrix-matrix multiplication. There is more reuse of weights:

$$Y = XW^T, \quad X \in \mathbb{R}^{S \times K}.$$

When S is large, arithmetic intensity increases and tensor core throughput matters more. In this case, activation precision and available tensor core instructions become more important.

Why INT4 May Not Automatically Be Four Times Faster

Although INT4 uses one quarter the memory of FP16, speedup depends on the bottleneck.

If the kernel is perfectly memory-bound and decode overhead is negligible, the ideal speedup from weight memory reduction is approximately:

$$\text{Speedup} \approx \frac{16}{4} = 4.$$

In practice:

$$\text{Speedup} < 4$$

because of:

- dequantization overhead,
- scale and zero-point memory traffic,
- imperfect coalescing,
- shared memory overhead,
- instruction limitations,
- dispatch overhead,
- cache effects.

When Fast Decoding Is Essential

Fast decoding matters especially when:

- the kernel uses 2-bit or 1-bit weights,
- the GPU is a data center GPU such as A100 or H100,
- the operation is small enough that decode overhead is visible,
- the theoretical memory savings are large but not realized due to scalar unpacking.

Why Layout Propagation Can Beat Local Optimization

Optimizing each operator independently can introduce extra layout conversions:

$$L_A \to L_B \to L_A \to L_C.$$

Each conversion can cost memory bandwidth and kernel launch overhead.

Graph-level layout propagation tries to keep tensors in a compatible layout across multiple operators:

$$L_A \to L_A \to L_A.$$

This is especially helpful for elementwise chains, residual adds, and patterns around quantized linear layers.

Common Misconceptions

Misconception 1: Low-Bit Storage Means Low-Bit Compute

A tensor stored as INT4 may still be computed using FP16, BF16, INT8, or another widened format after decoding. Storage precision and compute precision are separate design choices.

Misconception 2: Tensor Cores Automatically Handle Any Quantized Format

Tensor cores support specific instruction shapes and data type combinations. If the desired mixed precision operation is unsupported, the compiler must transform the operands into a compatible representation.

Misconception 3: Dequantization Location Is a Minor Detail

The location of dequantization changes global memory traffic, shared memory footprint, register pressure, and decode instruction overhead. It can determine whether a kernel approaches the expected speedup.

Misconception 4: Layout Is Only a Storage Detail

Layout is a performance-critical part of the computation. The same mathematical matrix can be fast or slow depending on how it is arranged for global memory, shared memory, and tensor core fragments.

Misconception 5: Dynamic Shapes Can Use One Universal Kernel

One kernel configuration is rarely optimal across all M, N, and K shapes. LLM inference benefits from segmented dispatch and tuning databases.

Practical Takeaways

1. Treat low-bit formats as packed storage plus explicit conversion logic.
2. Separate storage type, logical type, compute type, accumulator type, and output type.
3. For INT4, remember that one 32-bit word stores eight values.
4. For INT2, one 32-bit word stores sixteen values.
5. For 1-bit values, one 32-bit word stores thirty-two values.
6. Use fast vectorized decoding whenever decode overhead is visible.
7. In decode-stage LLM inference, weight bandwidth is often the bottleneck.
8. In prefill, tensor core utilization and activation precision may matter more.
9. Layout propagation is essential for avoiding unnecessary memory transformations.
10. Hardware-aligned layout inference is better than blindly searching all layouts.
11. Dynamic shape support requires dispatching to different tuned kernels.

12. A good low-bit compiler must reason across types, layouts, instructions, and graph structure.

Project or Experiment Ideas

Experiment 1: Naive Versus Vectorized INT4 Decode

Implement two CUDA kernels:

- scalar INT4 decode using shift and mask,
- vectorized INT4 decode that converts multiple values per packed word.

Measure effective decode bandwidth:

$$B_{\text{decode}} = \frac{\text{LogicalBytesDecoded}}{T}.$$

Experiment 2: Dequantization Location

Implement three variants of quantized GEMM:

1. dequantize weights into global memory before GEMM,
2. dequantize into shared memory inside GEMM,
3. dequantize into registers inside GEMM.

Compare runtime, shared memory usage, register usage, and achieved memory bandwidth.

Experiment 3: Decode Versus Prefill Shapes

Benchmark the same quantized linear layer for:

$$M = 1, \quad M = 16, \quad M = 128, \quad M = 4096.$$

Plot achieved throughput versus M. Expect small M to be more memory-bound and large M to become more compute-bound.

Experiment 4: Layout Swizzling

Write a tiled GEMM with two shared memory layouts:

- simple row-major shared memory,
- swizzled shared memory.

Use Nsight Compute to inspect shared memory bank conflicts and tensor core utilization.

Experiment 5: BitBLAS API Exploration

Use BitBLAS to generate kernels for:

$$\text{FP16} \times \text{INT4}, \quad \text{FP16} \times \text{INT2}, \quad \text{INT8} \times \text{INT4}.$$

Inspect generated CUDA source and compare the decode sections.

Final Mental Model

BitBLAS and Ladder are best understood as systems for making unsupported low-bit computation look hardware-native. The model wants flexible formats such as INT4, INT2, and 1-bit weights. The hardware wants fixed instruction types, fixed fragment layouts, coalesced global memory, bank-conflict-free shared memory, and carefully arranged register fragments.

The compiler bridge is:

packed custom data → layout-aware tiles → local conversion → hardware-supported instruction → graph-level

The main engineering lesson is that low precision performance is not obtained merely by storing fewer bits. It requires co-design across numerical format, bit packing, dequantization placement, memory layout, tensor core instruction selection, dynamic-shape dispatch, and graph-level fusion. BitBLAS provides the kernel-level machinery, Ladder provides the graph-level layout propagation, and Tile Language provides a more direct way to express these optimized kernels.